

Homework 2

Homework 2

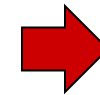
- To release today in Blackboard
- Covers Chapter 6-7
- Due: 11:59pm, Apr. 1
- No late homework will be accepted!

▼ 算法设计与分析

[主页](#)

[公告](#)

[Slides](#)



[Homework](#)

[Piazza](#)

Midterm

Time: Apr. 1 Tuesday, in class (1:00-2:40pm)

Location: TBA

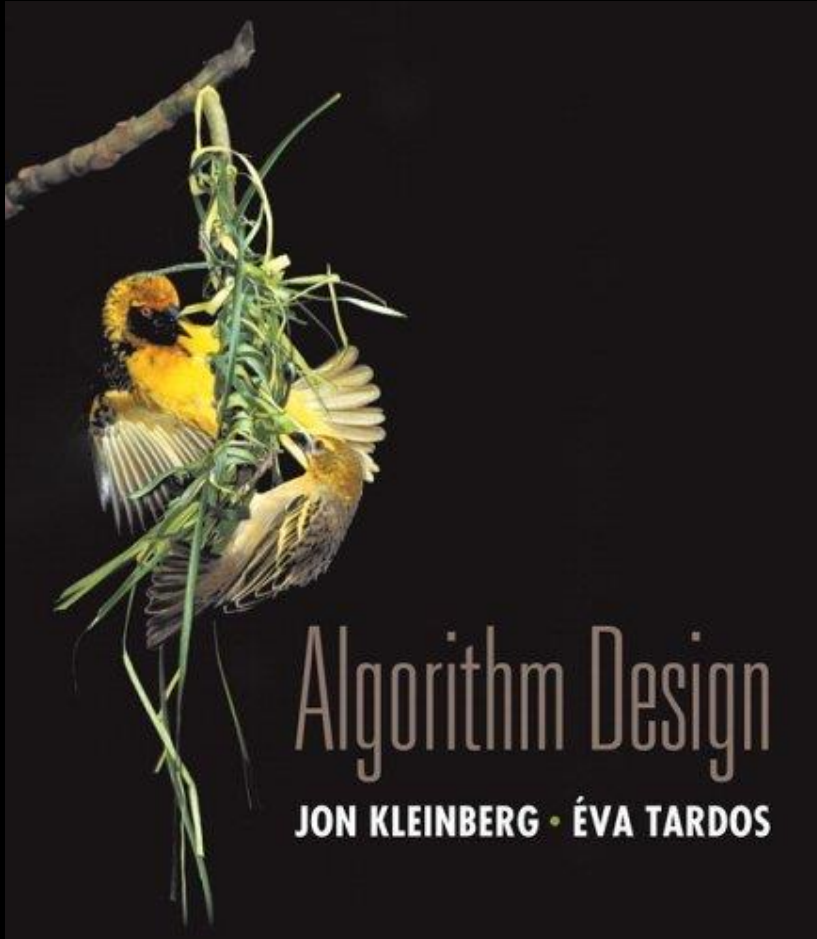
Covers Chapter 2~7

Format: 10 multi-choice questions and 4 problems; closed-book, one A4-size cheat sheet allowed

Grade: %35 of the total grade

Chapter 6

Dynamic Programming



Slides by Kevin Wayne.
Copyright © 2005 Pearson-Addison Wesley.
All rights reserved.

6.6 Sequence Alignment

String Similarity

How similar are two strings?

- **ocurrence**
- **occurrence**

o	c	u	r	r	a	n	c	e	-
o	c	c	u	r	r	e	n	c	e

6 mismatches, 1 gap

o	c	-	u	r	r	a	n	c	e
o	c	c	u	r	r	e	n	c	e

1 mismatch, 1 gap

o	c	-	u	r	r	-	a	n	c	e
o	c	c	u	r	r	e	-	n	c	e

0 mismatches, 3 gaps

Edit Distance

Edit distance. [Levenshtein 1966, Needleman-Wunsch 1970]

- Gap penalty δ ; mismatch penalty α_{pq} .
- Cost = sum of gap and mismatch penalties.
- Edit distance = min cost

C	T	G	A	C	C	T	A	C	C	T
---	---	---	---	---	---	---	---	---	---	---

-	C	T	G	A	C	C	T	A	C	C	T
---	---	---	---	---	---	---	---	---	---	---	---

C	C	T	G	A	C	T	A	C	A	T
---	---	---	---	---	---	---	---	---	---	---

C	C	T	G	A	C	-	T	A	C	A	T
---	---	---	---	---	---	---	---	---	---	---	---

$$\alpha_{TC} + \alpha_{GT} + \alpha_{AG} + 2\alpha_{CA}$$

$$2\delta + \alpha_{CA}$$

Applications.

- Basis for Unix diff.
- Speech recognition.
- Computational biology.

Sequence Alignment

Goal: Given two strings $X = x_1 x_2 \dots x_m$ and $Y = y_1 y_2 \dots y_n$ find alignment of minimum cost.

Def. An **alignment** M is a set of ordered pairs $x_i - y_j$ such that each item occurs in at most one pair and no crossings.

- The pair $x_i - y_j$ and $x_{i'} - y_{j'}$ **cross** if $i < i'$, but $j > j'$.

Def. The **cost** of an alignment

$$\text{cost}(M) = \underbrace{\sum_{(x_i, y_j) \in M} \alpha_{x_i y_j}}_{\text{mismatch}} + \underbrace{\sum_{i: x_i \text{ unmatched}} \delta + \sum_{j: y_j \text{ unmatched}} \delta}_{\text{gap}}$$

Ex: An alignment of CTACCG vs. TACATG.

$$M = x_2 - y_1, x_3 - y_2, x_4 - y_3, x_5 - y_4, x_6 - y_6.$$

x_1	x_2	x_3	x_4	x_5		x_6
C	T	A	C	C	-	G
-	T	A	C	A	T	G
	y_1	y_2	y_3	y_4	y_5	y_6

Sequence Alignment: Problem Structure

Def. $OPT(i, j)$ = min cost of aligning strings $x_1 x_2 \dots x_i$ and $y_1 y_2 \dots y_j$.

- Case 1: OPT matches x_i - y_j .
 - pay mismatch for x_i - y_j + min cost of aligning two strings $x_1 x_2 \dots x_{i-1}$ and $y_1 y_2 \dots y_{j-1}$
- Case 2a: OPT leaves x_i unmatched.
 - pay gap for x_i and min cost of aligning $x_1 x_2 \dots x_{i-1}$ and $y_1 y_2 \dots y_j$
- Case 2b: OPT leaves y_j unmatched.
 - pay gap for y_j and min cost of aligning $x_1 x_2 \dots x_i$ and $y_1 y_2 \dots y_{j-1}$

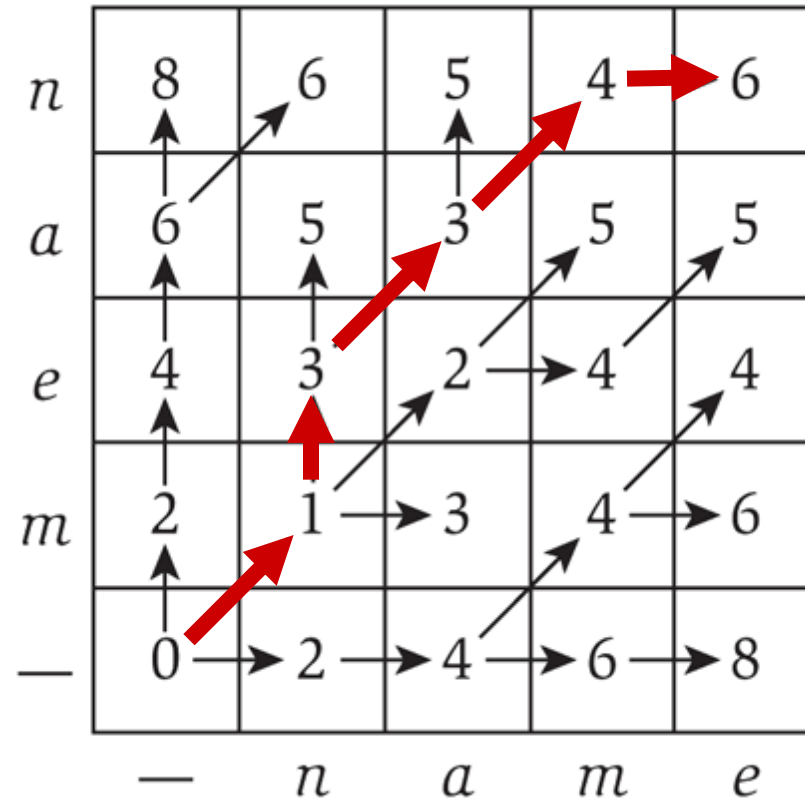
$$OPT(i, j) = \begin{cases} j\delta & \text{if } i = 0 \\ \min \begin{cases} \alpha_{x_i y_j} + OPT(i-1, j-1) \\ \delta + OPT(i-1, j) \\ \delta + OPT(i, j-1) \end{cases} & \text{otherwise} \\ i\delta & \text{if } j = 0 \end{cases}$$

Sequence Alignment: Algorithm

```
Sequence-Alignment( $m, n, x_1x_2\dots x_m, y_1y_2\dots y_n, \delta, \alpha$ ) {  
  for  $i = 0$  to  $m$   
     $M[0, i] = i\delta$   
  for  $j = 0$  to  $n$   
     $M[j, 0] = j\delta$   
  
  for  $i = 1$  to  $m$   
    for  $j = 1$  to  $n$   
       $M[i, j] = \min(\alpha[x_i, y_j] + M[i-1, j-1],$   
                     $\delta + M[i-1, j],$   
                     $\delta + M[i, j-1])$   
  
  return  $M[m, n]$   
}
```

Analysis. $\Theta(mn)$ time and space.

Sequence Alignment: Example



Sequence Alignment: Algorithm

Analysis. $\Theta(mn)$ time and space.

English words or sentences:

- $m, n \leq 30$. $\leftarrow$ OK

Computational biology:

- $m = n = 100,000$
- 10 billions ops is OK, but 10GB array is quite large

6.7 Sequence Alignment in Linear Space

Sequence Alignment: Linear Space

Q. Can we avoid using quadratic **space**?

Easy. Optimal **cost** in $O(m + n)$ space and $O(mn)$ time.

- Compute $\text{OPT}(i, \cdot)$ from $\text{OPT}(i-1, \cdot)$.
- No longer a simple way to recover alignment itself.

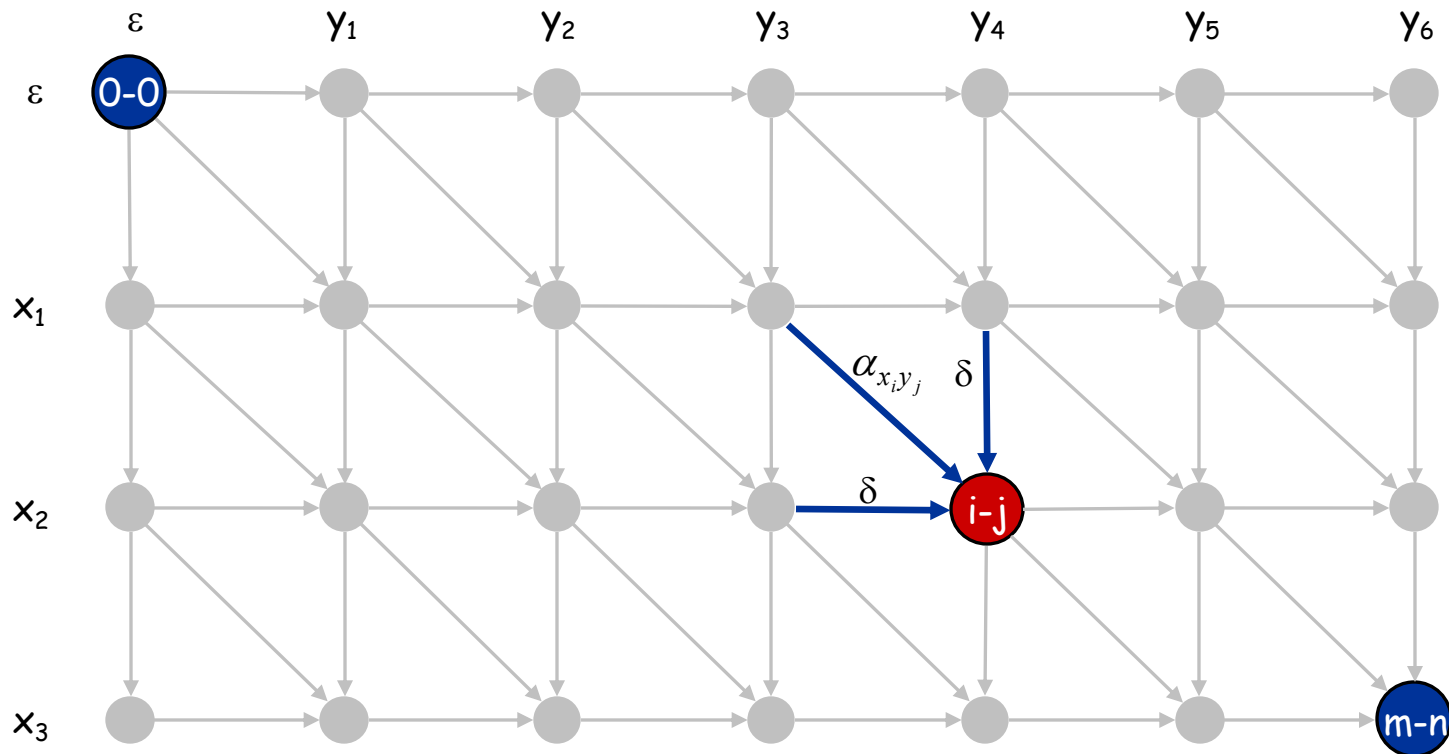
Theorem. [Hirschberg 1975] Optimal **alignment** in $O(m + n)$ space and $O(mn)$ time.

- Clever combination of divide-and-conquer and dynamic programming.

Sequence Alignment: Linear Space

Edit distance graph.

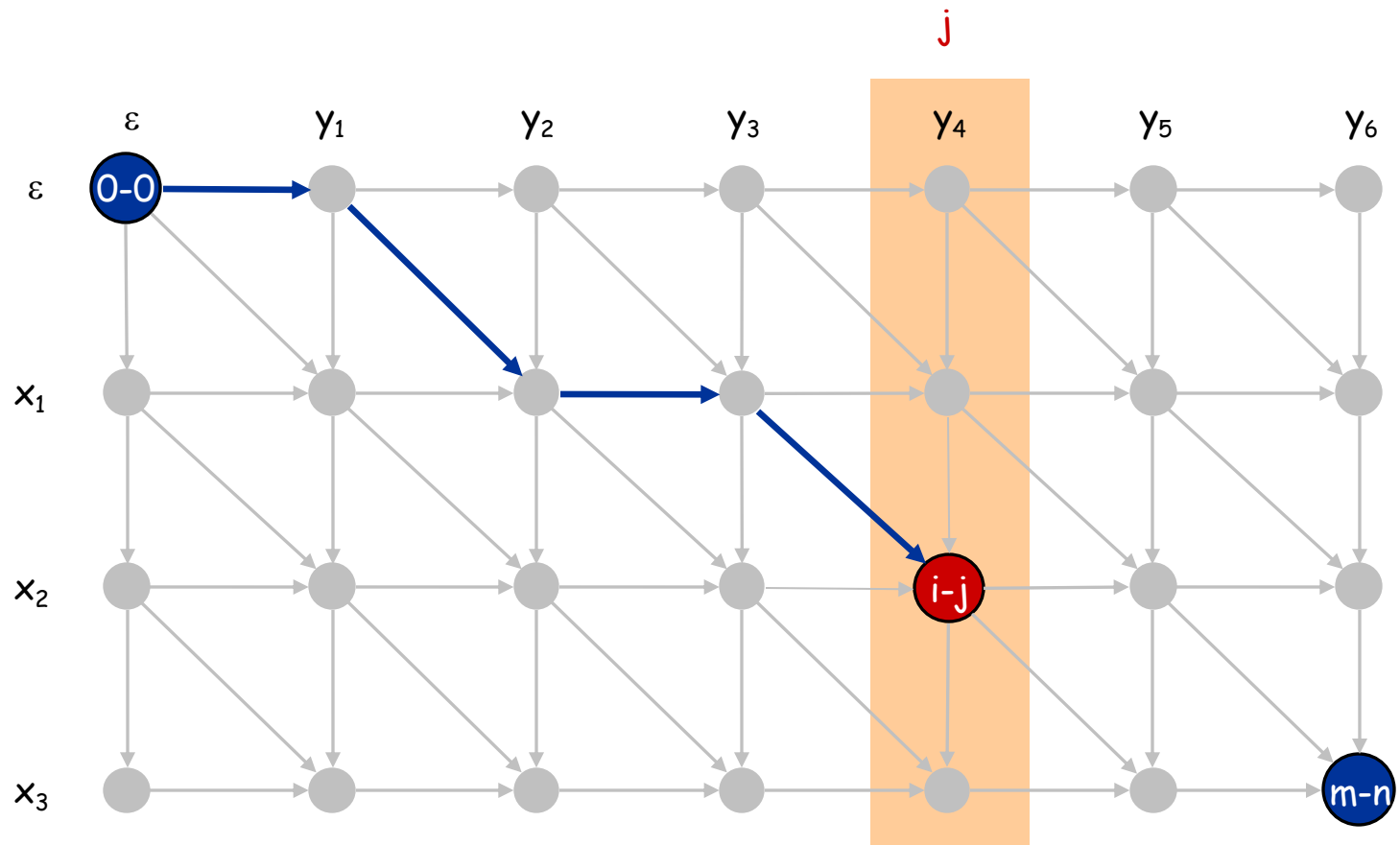
- Let $f(i, j)$ be shortest path from $(0,0)$ to (i, j) .
- Observation: $f(i, j) = \text{OPT}(i, j)$.



Sequence Alignment: Linear Space

Edit distance graph.

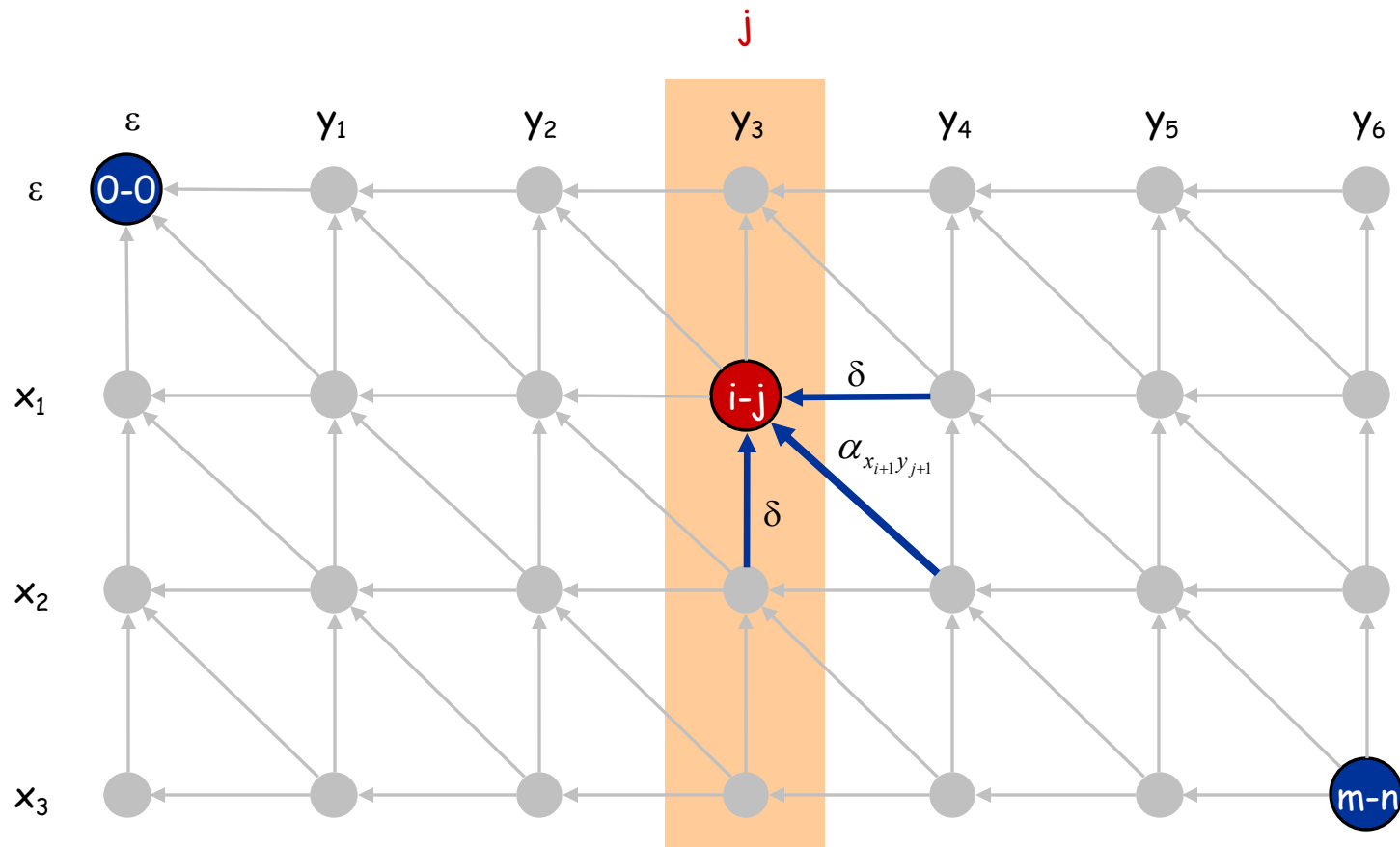
- Let $f(i, j)$ be shortest path from $(0,0)$ to (i, j) .
- Can compute $f(\cdot, j)$ for any j in $O(mn)$ time and $O(m + n)$ space.



Sequence Alignment: Linear Space

Edit distance graph.

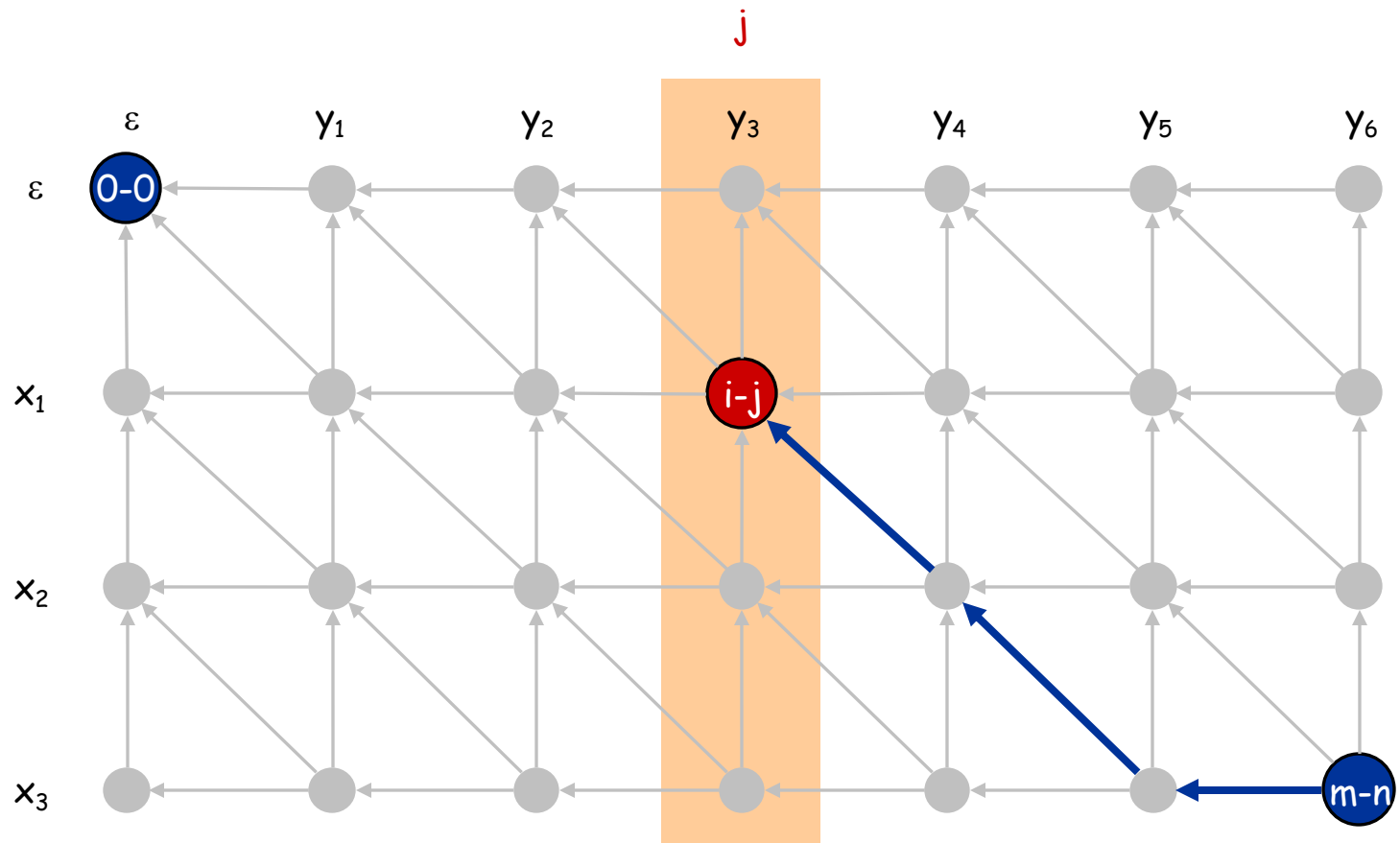
- Let $g(i, j)$ be shortest path from (i, j) to (m, n) .
- Can compute $g(\cdot, j)$ by reversing the edge orientations and inverting the roles of $(0, 0)$ and (m, n)



Sequence Alignment: Linear Space

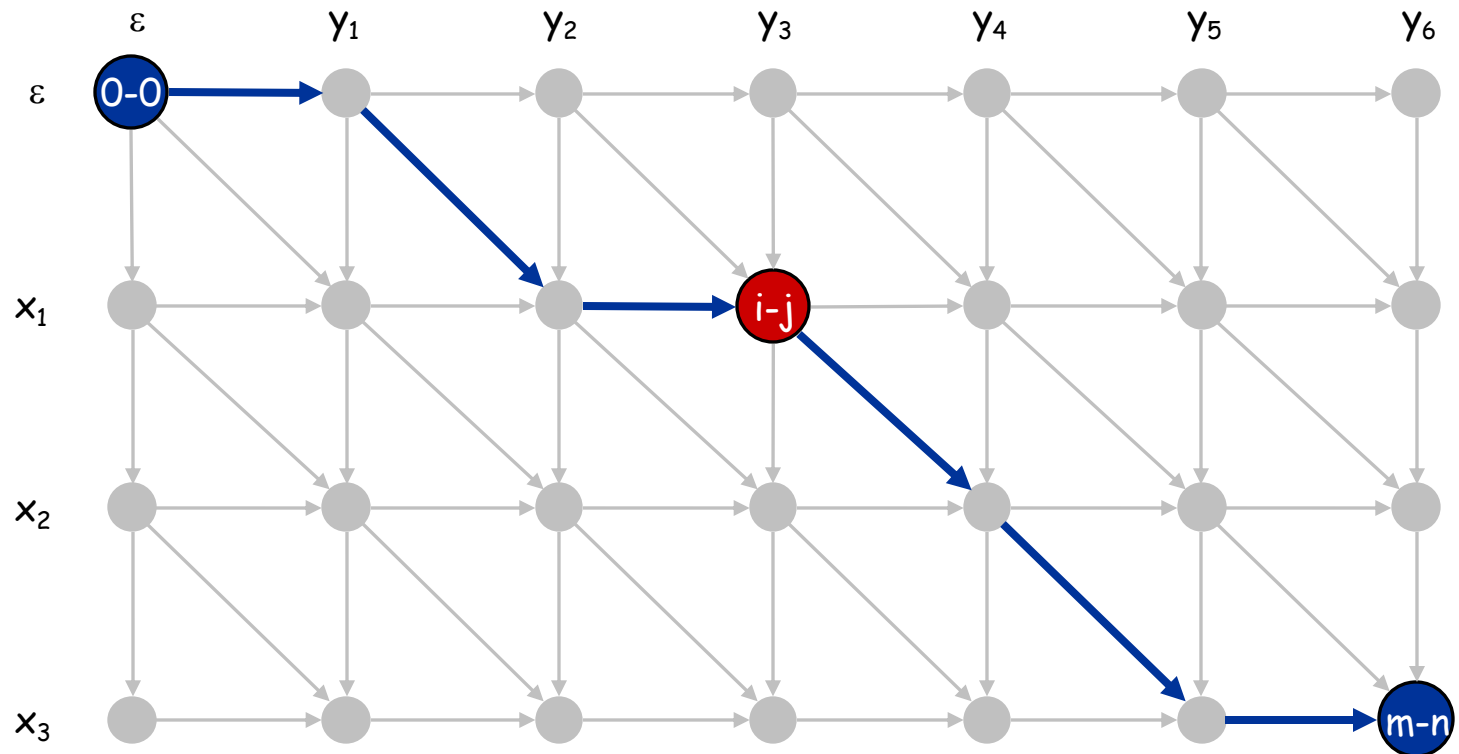
Edit distance graph.

- Let $g(i, j)$ be shortest path from (i, j) to (m, n) .
- Can compute $g(\cdot, j)$ for any j in $O(mn)$ time and $O(m + n)$ space.



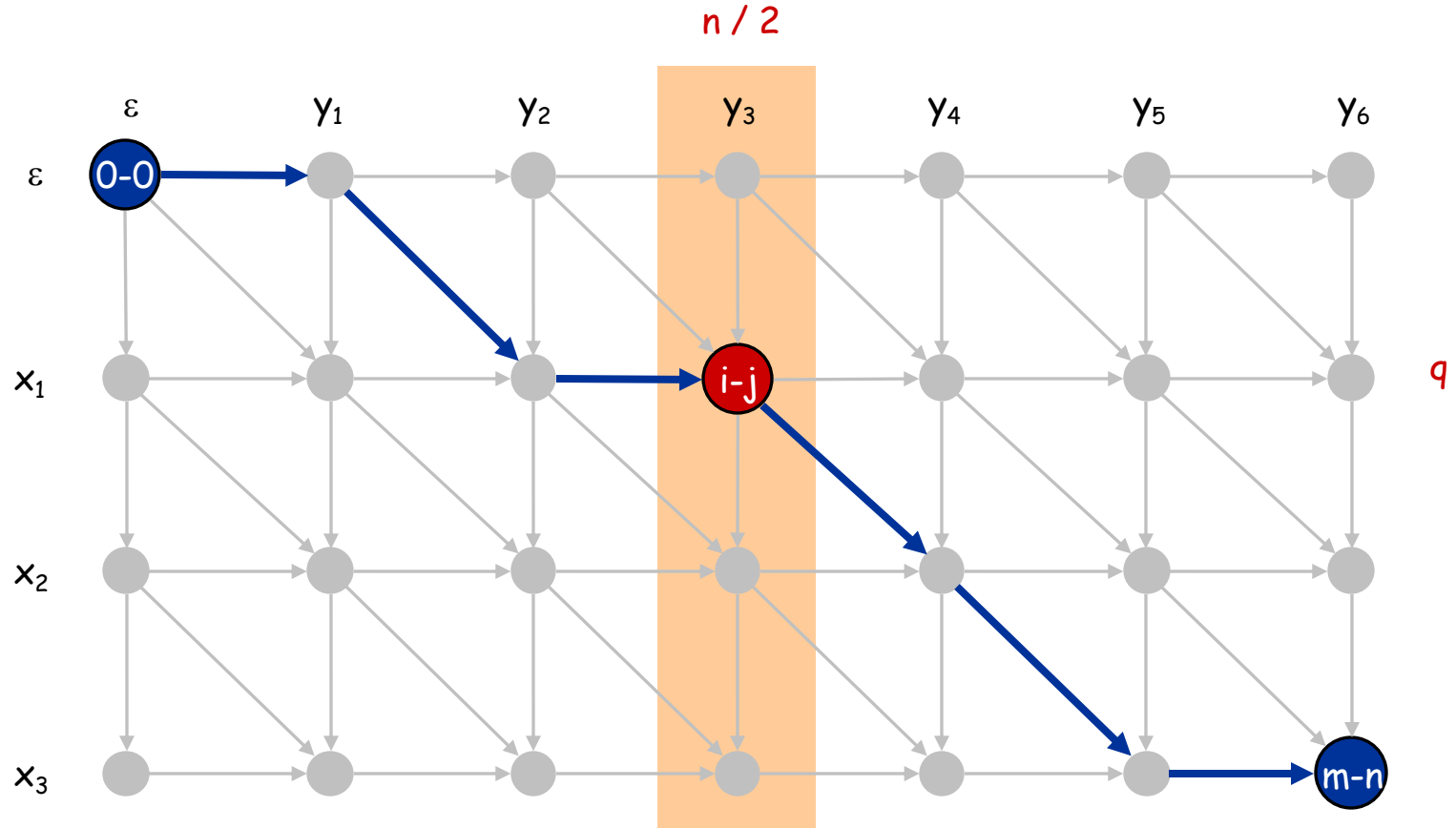
Sequence Alignment: Linear Space

Observation 1. The cost of the shortest path that uses (i, j) is $f(i, j) + g(i, j)$.



Sequence Alignment: Linear Space

Observation 2. let q be an index that minimizes $f(q, n/2) + g(q, n/2)$. Then, the shortest path from $(0, 0)$ to (m, n) uses $(q, n/2)$.

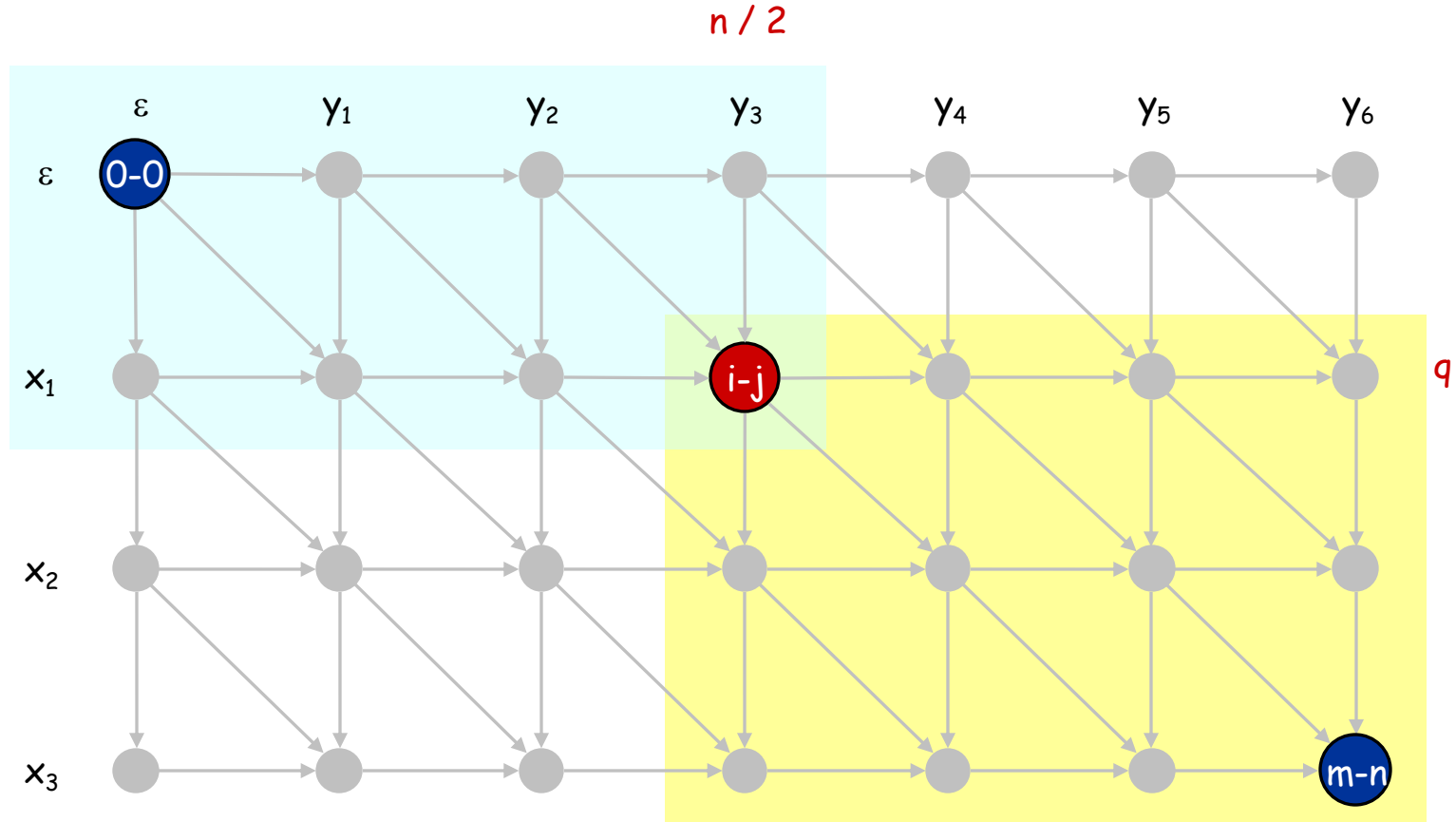


Sequence Alignment: Linear Space

Divide: find index q that minimizes $f(q, n/2) + g(q, n/2)$ using DP.

▫ Do alignment at $(x_q, y_{n/2})$.

Conquer: recursively compute optimal alignment in each piece.



Sequence Alignment: Running Time Analysis

Theorem. Let $T(m, n)$ = max running time of algorithm on strings of length m and n . $T(m, n) = O(mn)$.

Pf. (by induction on n)

- $O(mn)$ time to compute $f(\cdot, n/2)$ and $g(\cdot, n/2)$ and find index q .
- $T(q, n/2) + T(m - q, n/2)$ time for two recursive calls.
- Choose constant c so that:

$$T(m, 2) \leq cm$$

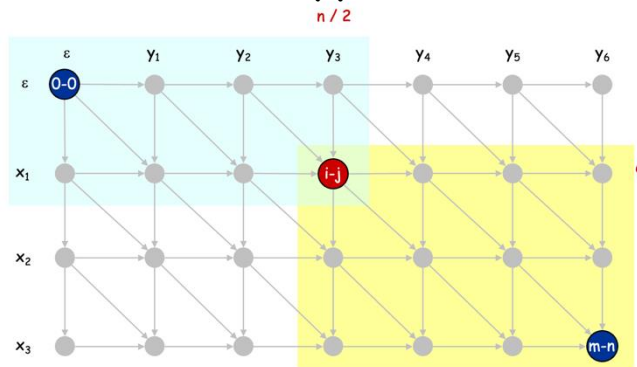
$$T(2, n) \leq cn$$

$$T(m, n) \leq cmn + T(q, n/2) + T(m - q, n/2)$$

- Claim: $T(m, n) \leq 2cmn$

- Base cases: $m = 2$ or $n = 2$.

- Inductive hypothesis: $T(m', n') \leq 2cm'n'$ with $m' < m$ and $n' < n$



$$\begin{aligned} T(m, n) &\leq T(q, n/2) + T(m - q, n/2) + cmn \\ &\leq 2cq n/2 + 2c(m - q)n/2 + cmn \\ &= cq n + cmn - cq n + cmn \\ &= 2cmn \end{aligned}$$

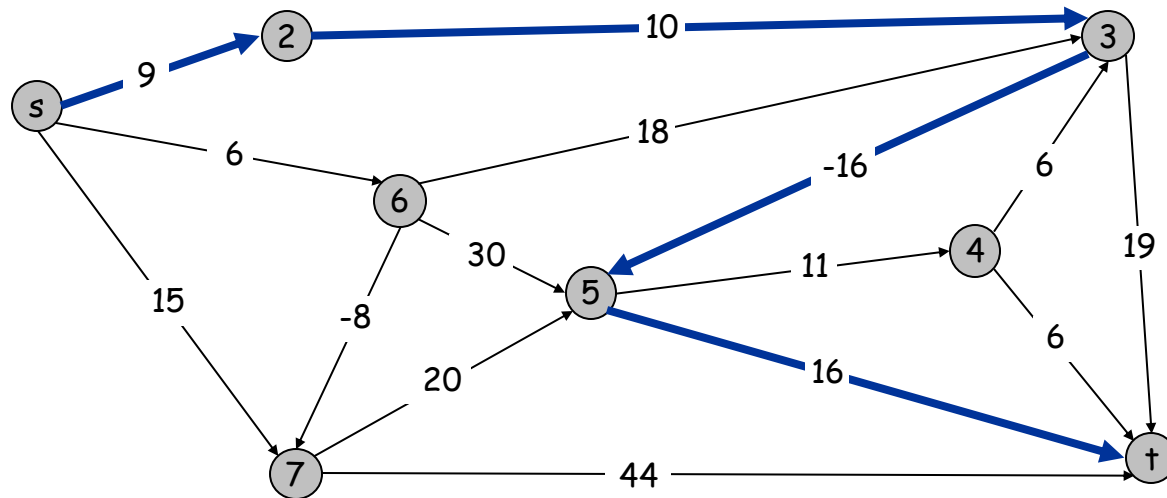
6.8 Shortest Paths

Shortest Paths

Shortest path problem. Given a directed graph $G = (V, E)$, with edge weights c_{vw} , find shortest path from node s to node t .

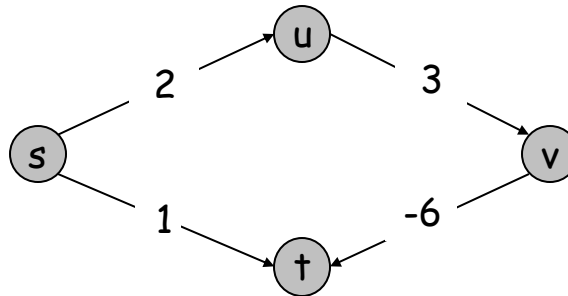
↖ allow negative weights

Ex. Nodes represent agents in a financial setting and c_{vw} is cost of transaction in which we buy from agent v and sell immediately to w .

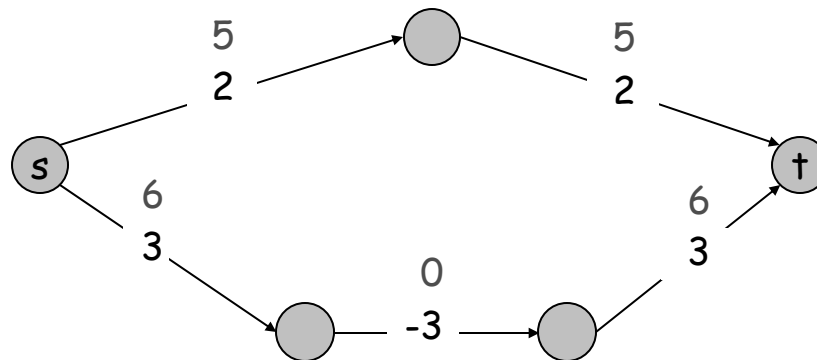


Shortest Paths: Failed Attempts

Dijkstra. Can fail if negative edge costs.

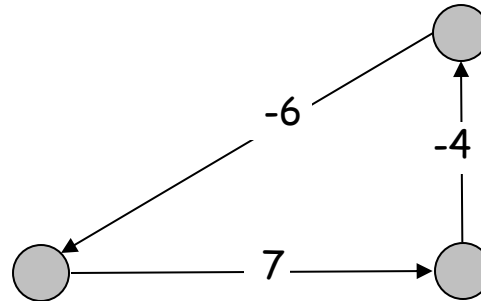


Re-weighting. Adding a constant to every edge weight can fail.

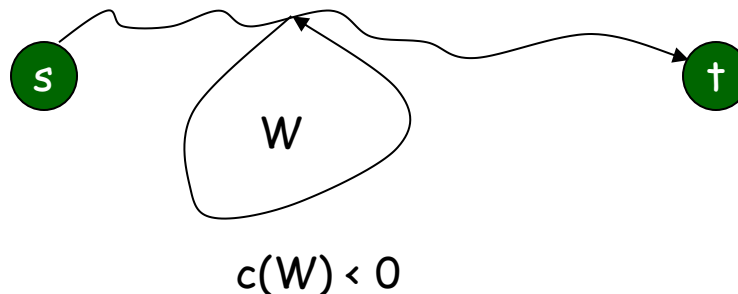


Shortest Paths: Negative Cost Cycles

Negative cost cycle.



Observation. If some path from s to t contains a negative cost cycle, there does not exist a shortest s - t path; otherwise, there exists one that is simple.



Shortest Paths: Dynamic Programming

Def. $OPT(i, v)$ = length of shortest v - t path P using at most i edges.

- Case 1: P uses at most $i-1$ edges.
 - $OPT(i, v) = OPT(i-1, v)$
- Case 2: P may use i edges.
 - if (v, w) is first edge, then OPT uses (v, w) , and then selects best w - t path using at most $i-1$ edges

$$OPT(i, v) = \begin{cases} \infty & \text{if } i = 0, v \neq t \\ 0 & \text{if } v = t \\ \min \left\{ OPT(i-1, v), \min_{(v, w) \in E} \{ OPT(i-1, w) + c_{vw} \} \right\} & \text{otherwise} \end{cases}$$

Remark. By previous observation, if no negative cycles, then $OPT(n-1, v)$ = length of shortest v - t path.

Shortest Paths: Implementation

```
Shortest-Path( $G, s, t$ ) {  
    foreach node  $v \in V$   
         $M[0, v] \leftarrow \infty$   
     $M[0, t] \leftarrow 0$   
  
    for  $i = 1$  to  $n-1$   
        foreach node  $v \in V$   
             $M[i, v] \leftarrow M[i-1, v]$   
            foreach edge  $(v, w) \in E$   
                 $M[i, v] \leftarrow \min \{ M[i, v], M[i-1, w] + c_{vw} \}$   
  
    return  $M[n-1, s]$   
}
```

Analysis. $\Theta(mn)$ time, $\Theta(n^2)$ space.

Finding the shortest paths. Maintain a "successor" for each table entry.

Shortest Paths: Improvements

```
Shortest-Path( $G, s, t$ ) {  
    foreach node  $v \in V$   
         $M[0, v] \leftarrow \infty$   
     $M[0, t] \leftarrow 0$   
  
    for  $i = 1$  to  $n-1$   
        foreach node  $v \in V$   
             $M[i, v] \leftarrow M[i-1, v]$   
            foreach edge  $(v, w) \in E$   
                 $M[i, v] \leftarrow \min \{ M[i, v], M[i-1, w] + c_{vw} \}$   
  
    return  $M[n-1, s]$   
}
```

Practical improvements.

- Maintain only one array $M[v]$ = shortest v - t path that we have found so far.

Shortest Paths: Improvements

```
Shortest-Path( $G, s, t$ ) {  
  foreach node  $v \in V$   
     $M[v] \leftarrow \infty$   
   $M[t] \leftarrow 0$   
  
  for  $i = 1$  to  $n-1$   
    foreach node  $v \in V$   
     $M[v] \leftarrow M[v]$   
    foreach edge  $(v, w) \in E$   
       $M[v] \leftarrow \min \{ M[v], M[w] + c_{vw} \}$   
  
  return  $M[s]$   
}
```

Practical improvements.

- Maintain only one array $M[v]$ = shortest v - t path that we have found so far.
- No need to check edges of the form (v, w) unless $M[w]$ changed in previous iteration.

Bellman-Ford: Efficient Implementation

```
Push-Based-Shortest-Path( $G, s, t$ ) {  
    foreach node  $v \in V$   
         $M[v] \leftarrow \infty$   
     $M[t] \leftarrow 0$   
  
    for  $i = 1$  to  $n-1$  {  
        foreach node  $w \in V$   
            if ( $M[w]$  has been updated in previous iteration)  
                foreach node  $v$  such that  $(v, w) \in E$   
                    if ( $M[v] > M[w] + c_{vw}$ )  
                         $M[v] \leftarrow M[w] + c_{vw}$   
                If no  $M[w]$  value changed in iteration  $i$ , stop.  
    }  
  
    return  $M[s]$   
}
```

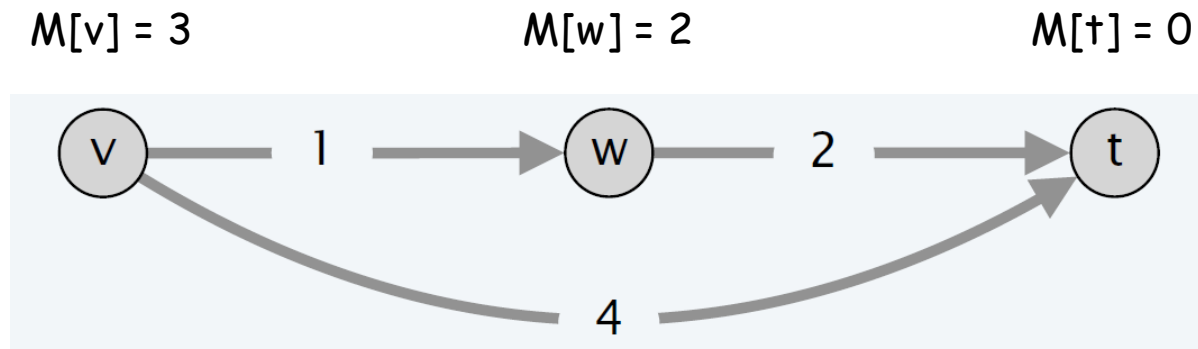
Analysis.

- $O(n)$ extra space
- Time: $O(mn)$ worst case, but substantially faster in practice.

Bellman-Ford: Efficient Implementation

Claim. Throughout the algorithm, $M[v]$ is length of some v - t path, and after i rounds of updates, the value $M[v]$ is the length of shortest v - t path using $\leq i$ edges.

Counter-example:



if nodes w considered before node v ,
then $M[v] = 3$ after 1 pass

Bellman-Ford: Efficient Implementation

Claim. Throughout the algorithm, $M[v]$ is length of some v - t path, and after i rounds of updates, the value $M[v]$ is **no larger than** the length of shortest v - t path using $\leq i$ edges.

6.9 Distance Vector Protocol

Distance Vector Protocol

Communication network.

- Nodes $\approx$ routers.
- Edges $\approx$ direct communication link.
- Cost of edge $\approx$ delay on link. $\leftarrow$ naturally nonnegative

Dijkstra's algorithm. Requires global information of network.

Bellman-Ford. Uses only local knowledge of neighboring nodes.

Synchronization. We don't expect routers to run in lockstep. The order in which each `foreach` loop executes is not important. Moreover, algorithm still converges even if updates are asynchronous.

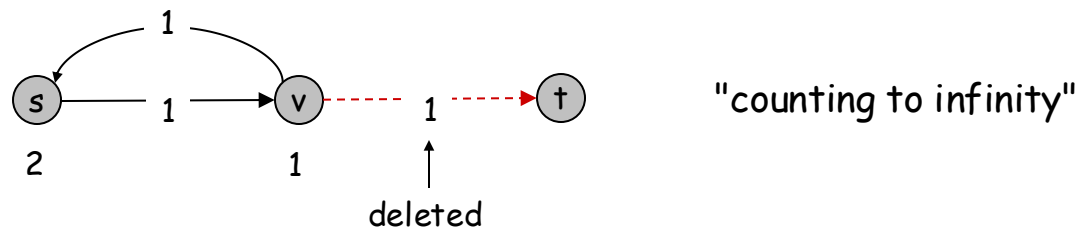
Distance Vector Protocol

Distance vector protocol.

- Each router maintains a vector of shortest path lengths to every other node (distances) and the first hop on each path (directions).
- Algorithm: each router performs n separate computations, one for each potential destination node.
- "Routing by rumor."

Ex. RIP, Xerox XNS RIP, Novell's IPX RIP, Cisco's IGRP, DEC's DNA Phase IV, AppleTalk's RTMP.

Caveat. Edge costs may **change** during algorithm (or fail completely).



Path Vector Protocols

Link state routing.

- Each router also stores the entire path.
- Avoids "counting-to-infinity" problem and related difficulties.
- Requires significantly more storage.

not just the distance and first hop

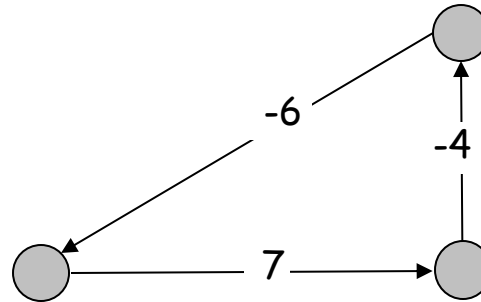


Ex. Border Gateway Protocol (BGP), Open Shortest Path First (OSPF).

6.10 Negative Cycles in a Graph

Detecting Negative Cycles

Negative cycle detection problem. Given a digraph $G = (V, E)$, with edge weights c_{vw} , find a negative cycle (if one exists).

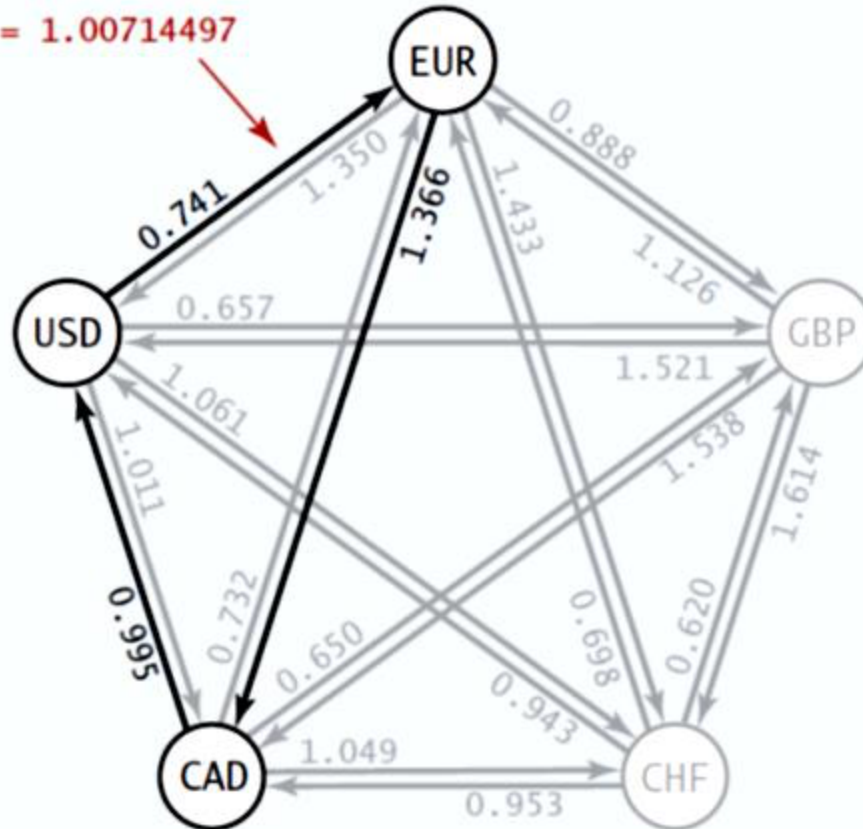


Detecting Negative Cycles: Application

Currency conversion. Given n currencies and exchange rates between pairs of currencies, is there an arbitrage opportunity?

Remark. Fastest algorithm very valuable!

$$0.741 * 1.366 * .995 = 1.00714497$$

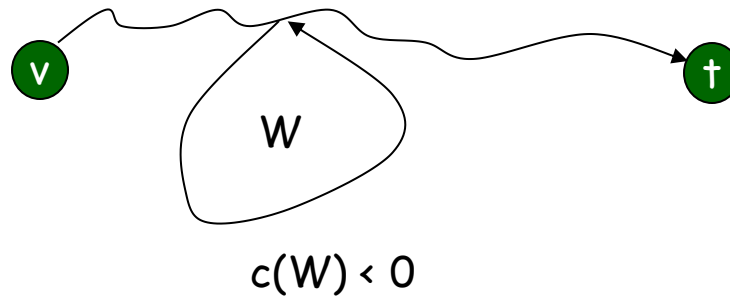


Detecting Negative Cycles

Lemma. If $\text{OPT}(n,v) = \text{OPT}(n-1,v)$ for all v , then there is no negative cycle with a path to t .

Pf. (by contradiction)

- $\text{OPT}(n,v) = \text{OPT}(n-1,v) \Rightarrow \text{OPT}(i,v) = \text{OPT}(n-1,v)$ for $i \geq n$
- But negative cycle in a path implies that $\text{OPT}(i,v)$ always decreases as i increases

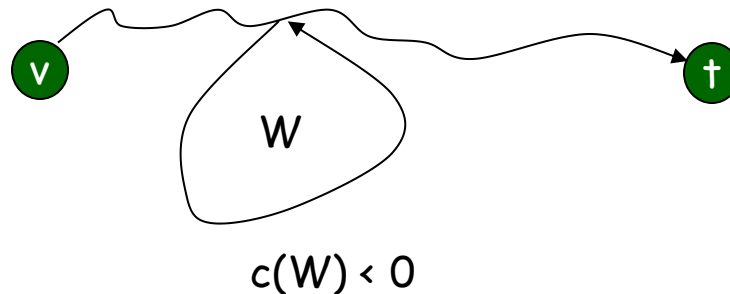


Detecting Negative Cycles

Lemma. If $\text{OPT}(n,v) < \text{OPT}(n-1,v)$ for some node v , then (any) shortest path from v to t contains a cycle W . Moreover W has negative cost.

Pf. (by contradiction)

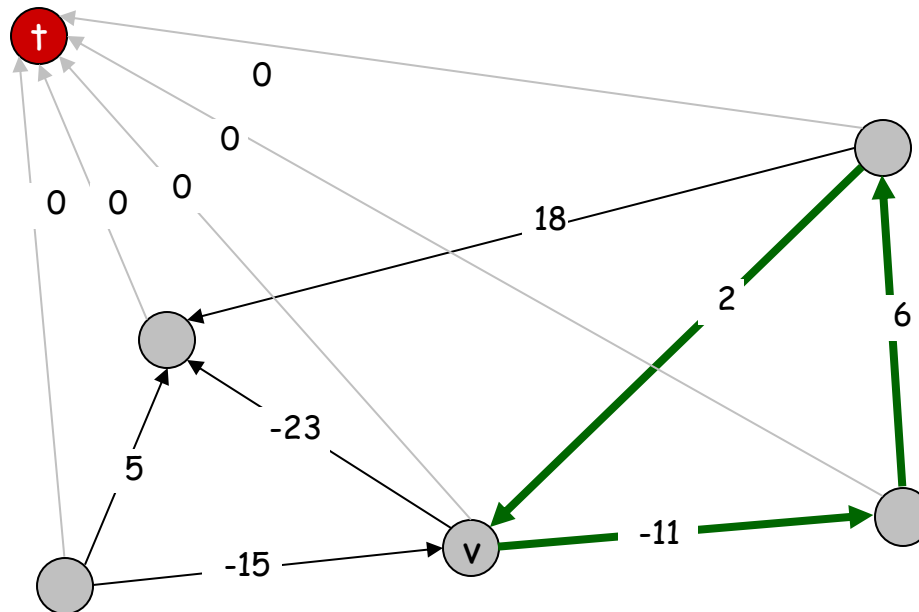
- Since $\text{OPT}(n,v) < \text{OPT}(n-1,v)$, we know the shortest v - t path P has exactly n edges. ← For n nodes, only $(n-1)$ edges needed for an acyclic path.
- By pigeonhole principle, P must contain a directed cycle W .
- Deleting W yields a v - t path with $< n$ edges $\Rightarrow W$ has negative cost.
← Assume P' is the path deleted W , $\text{cost}(P') \geq \text{OPT}(n-|W|,v) \geq \text{OPT}(n-1,v)$.



Detecting Negative Cycles

Theorem. Can detect negative cost cycle in $O(mn)$ time.

- Add new node t and connect all nodes to t with 0-cost edge.
- Check if $\text{OPT}(n, v) = \text{OPT}(n-1, v)$ for all nodes v .
 - if yes, then no negative cycles
 - if no, then extract cycle from shortest path from v to t



Dynamic Programming: Chapter Summary

Dynamic Programming

Basic idea

- Polynomial number of sub-problems with a natural ordering from smallest to largest.
- Optimal solution to a sub-problem can be constructed from optimal solutions of smaller sub-problems.
- Sub-problems are overlapping!

Guideline

- Define the sub-problems
 - $OPT(\dots)$
- Write down the recursive formulas
 - Ex: $OPT(i) = \max(f(OPT(j)), g(OPT(k)), \dots), j, k < i$
- Compute the formulas either bottom-up or top-down

Dynamic Programming

Algorithms

- Weighted interval scheduling
 - 1D array; binary choice
- Knapsack
 - 2D array; adding a new variable (weight limit)
- RNA secondary structure
 - 2D array: intervals
- Sequence Alignment
 - 2D array: prefix alignment
- Sequence Alignment in Linear Space
 - Combination of divide-and-conquer and dynamic programming
- Shortest path with negative edges
 - (Bellman-Ford) 2D array: shortest path with edge number $\leq i$
- Distance Vector Protocol
- Negative Cycle Detection